

BTAF Token (BTAF) SMART CONTRACT Security Audit

Performed on Contract:

BTAFToken.sol:
0xcAE3d82D63e2b0094bc959752993D3D3743B5D08

Platform
BSC

hashlock.com.au

JANUARY 2023

Table of Contents

Executive Summary	4
Project Context	4
Roadmap	7
Audit scope	8
Security Rating	9
Standardised Checks	10
Intended Smart Contract Functions	12
Function List	12
Code Quality	13
Audit Resources	13
Dependencies	13
Severity Definitions	14
Audit Findings	15
Centralisation	16
Conclusion	17
Our Methodology	18
Disclaimers	20
About Hashlock	21
Appendix	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

Bitcoin Trade and Forecast's (BTAF) team partnered with Hashlock to conduct a security audit of their BNB Smart Chain (BSC) smart contract after their Token's launch. Hashlock manually and proactively reviewed the code in order to ensure the project was safe for continued use and to give confidence to their community.

Project Context

[BTAF token](#) is the primary token to be used on [BitcoinTAF's](#) main website to purchase products and services via their BTAF Pay solution. BitcoinTAF is a leading platform for cryptocurrency trading education, technical data, and fundamental research. See the whitepaper [here](#) for more context.

Token Name: BTAF Token (BTAF)

Contract Address: [0xcAE3d82D63e2b0094bc959752993D3D3743B5D08](#)

Total Supply: 200,000,000

Compiler Version: v0.8.17+commit.8df45f5f

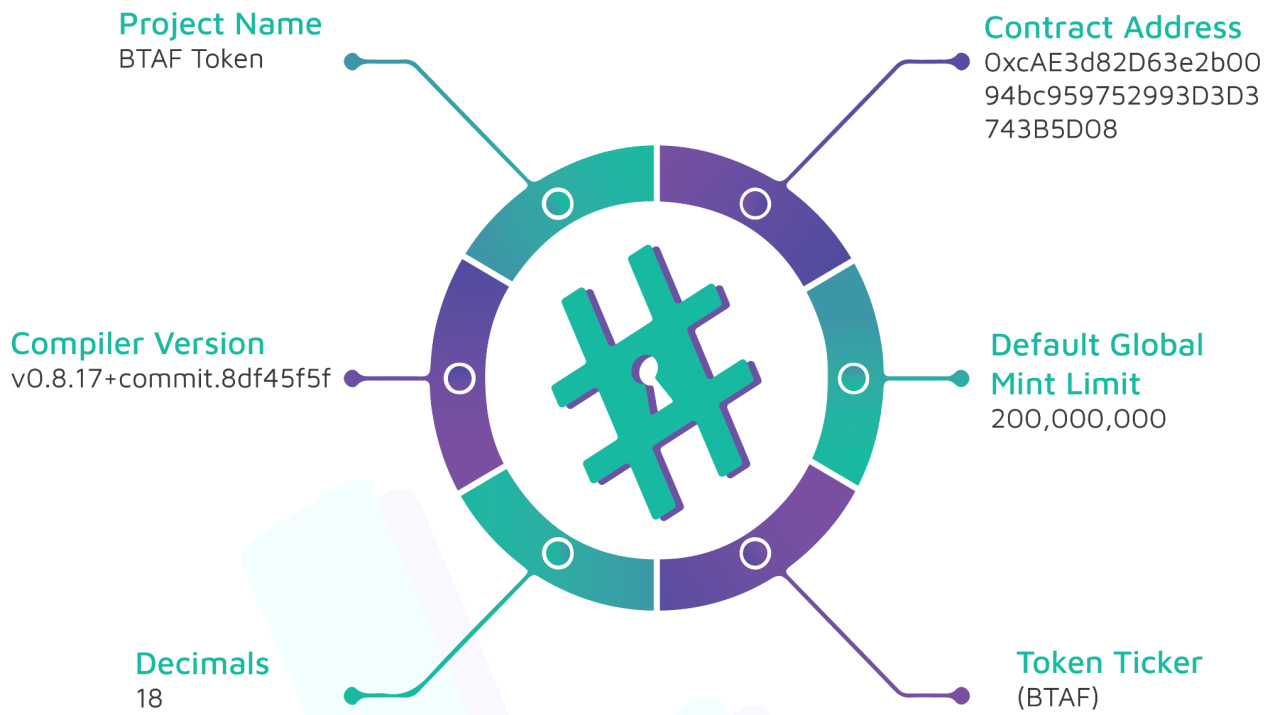
Decimals: 18

Logo:



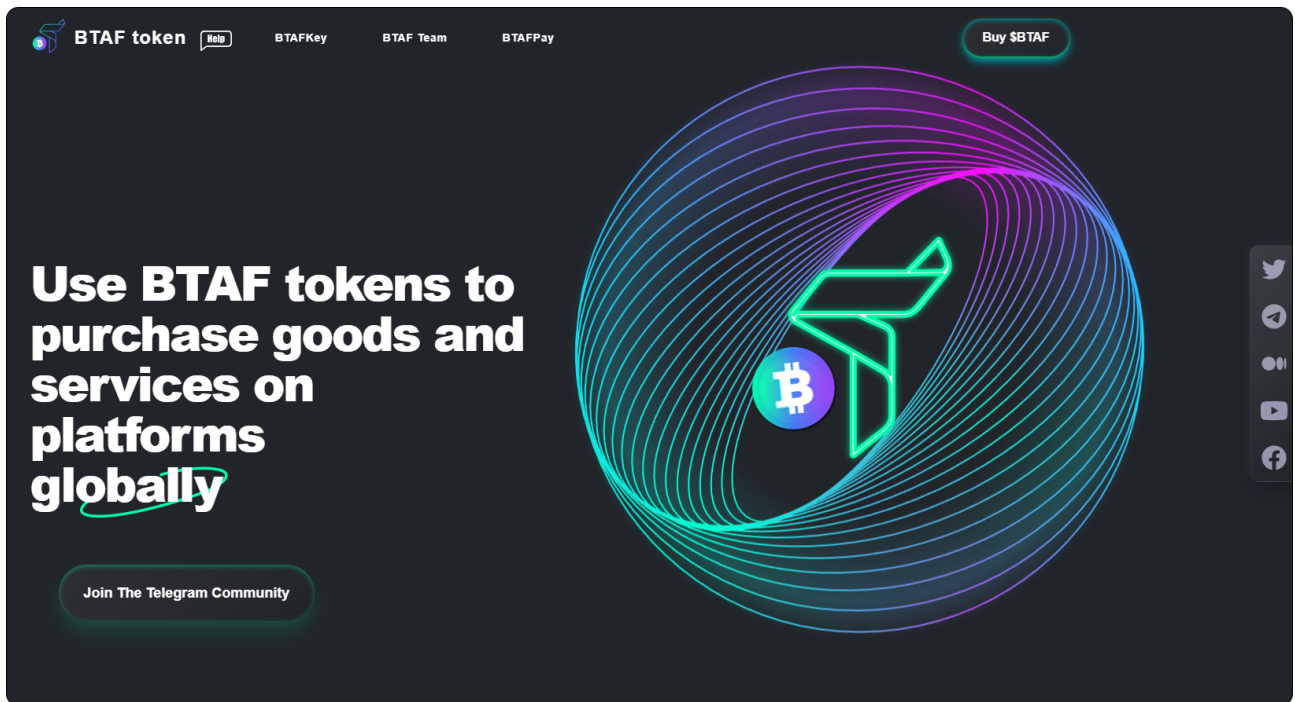
Mascot: (Tuffy)



Visualised Context:

Platform Visuals:

BTAF token website:



BitcoinTAF website:



Roadmap

Launch:

- #BTAF token
- Deploy Governance
- Deploy on Pancake Swap

Q4 2022:

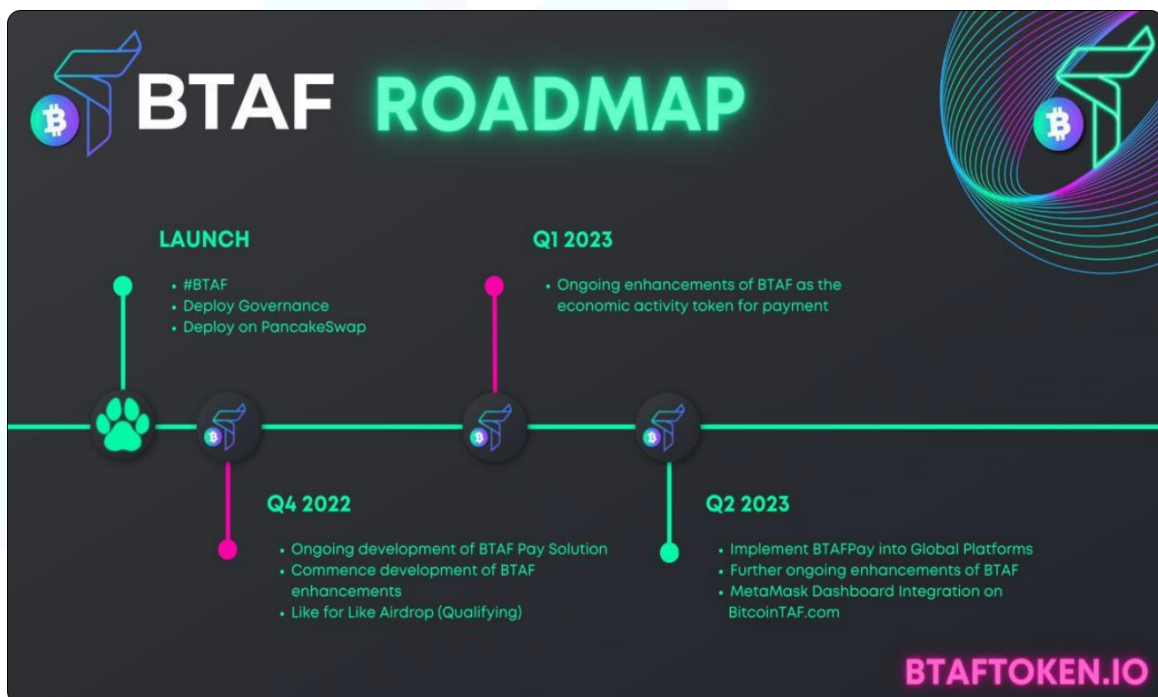
- Ongoing development of BTAF Pay Solution
- Commence development of BTAF enhancements
- Like for Like Airdrop (Qualifying)

Q1 2023:

- Ongoing enhancements of BTAF as the economic activity token for payment

Q2 2023:

- Implement BTAFPay into Global Platforms, further ongoing enhancements of BTAF Metamask dashboard integration on BitcoinTAF.com



Audit scope

We at Hashlock audited the solidity code within the BTAF Token, the scope of works included a comprehensive review of the smart contract listed below.

We tested the smart contract to check its security and efficiency. These tests were undertaken through both manual and software assisted analysis.

Description	Project Review and Security Analysis Report for BTAF Token Smart Contract.
Platform	BSC / Solidity
Audit Date	Jan 9th, 2023
Contract Address	0xcAE3d82D63e2b0094bc959752993D3D3743B5D08
Contract	BTAFToken.sol
Contract MD5 Hash	71445569137b96c9b191a4e43f3f0479

Security Rating

After our audit and analysis, we found the smart contract to be “**Secure**”. The contract follows simple logic, with correct and detailed ordering. The contract employs a series of interfaces for operations and the protocol utilises a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for fully decentralised, highly secure projects with little owner influence.

NOTE: The BSCScan [link](#) also includes a variety of old contracts unintentionally deployed, that do not interact with the BTAF token. The relevant contracts to this audit are: BTAFToken.sol, ERC20PresetMinterPauser.sol, and Ownable.sol. The old contracts at the same address do have vulnerabilities in them however, so these must never be used or called. The old deployed contracts cannot cause issues with the BTAF token.

Our primary concern was to identify arithmetic or logic vulnerabilities that may allow for funds to be lost, and to evaluate owner access control.

There does not seem to be any arithmetic or logic errors that affect security.

All issues uncovered during analysis were manually reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

Caution: *Hashlock's audit's do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity / compiler version stated	Passed
	Consistent pragma version across each contract	Reviewed
	Outdated Solidity Version	Passed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Passed
	Lack of check on input parameters	Passed
	Function input parameters check bypass	Passed
	Correct Access control	Passed
	Built in emergency features	Passed
	Correct event logs	Passed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerabilities	Passed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed
	High consumption 'storage' storage	Passed

	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Reviewed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Overall Audit Result: PASSED

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Contract: BTAFToken.sol - Name: BTAF - Symbol: BTAF - Decimals: 18 - Total Supply: 200 Million	Contract achieves this functionality.

Function List

BTAFToken.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	Mint	write	Owner can mint unlimited tokens	Refer Audit Findings
3	pause	write	Passed	No Issue
4	unpause-	write	Passed	No Issue
5	_beforeTokenTransfer	internal	Passed	No Issue
6	mint	write	Passed	No Issue

Code Quality

This audit scope involves the solidity smart contract of the BTAFToken, as outlined in the Audit Scope section. The contract, libraries and interfaces follow standard best practices and avoid unnecessary complexity that increases the likelihood of exploitation.

Majority of the code is well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Some minor opportunities of improvement are:

- Somewhat Inconsistent solidity versioning
- Not the most recent solidity version

Audit Resources

We were given the BTAF smart contract code in the form of mainnet links after deployment.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are very helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contract infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
Critical	Critical vulnerabilities are of immediate concern, and allow hackers or malicious code to steal funds, take ownership of the contract, or other devastating actions.
High	High vulnerabilities are not attacked as easily, but still have significant implications on the protocol.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Informational	These are mostly preferred factors that won't affect security.

Audit Findings

Critical

No critical vulnerabilities were found.

High

No high-level vulnerabilities were found.

Medium

1) - **ERC20PresetMinterPauser.sol**

The Minter role can mint unlimited tokens. There is no mint / supply limit and so tokens can be minted to any account.

Resolution: There should be a limit for minting or need to confirm. If this is intended, then disregard this issue. After talking to the project team, they intend to use this function for controlled inflation in the future.

The BTAF team has Acknowledged this.

Low

No low-level vulnerabilities were found.

Very Low / Informational / Best practices:

- 1) Token does not use the most recent version of solidity, and the version is not consistent with its dependencies. The version used is recent however.

The BTAF team has Acknowledged this.

Centralisation

The owner executable functions within the protocol can increase security but depend exclusively on private key security.

This level of centralisation is suitable for a community based project with a public leadership team.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, BTAFToken follows standard, best practice structure and is protected. Overall, the code is correctly ordered and follows industry best practices.

Stakeholders just need to note the potential for unlimited minting of tokens (which is common), and the dependance on owners for personal security of private keys and activity. The BTAF team intends to use this function for controlled inflation in the future.

As this audit occurred after launch, our suggestions cant easily be resolved.

Hashlock believes the BTAFToken solidity code base is sufficiently protected.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the smart contract in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contract.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

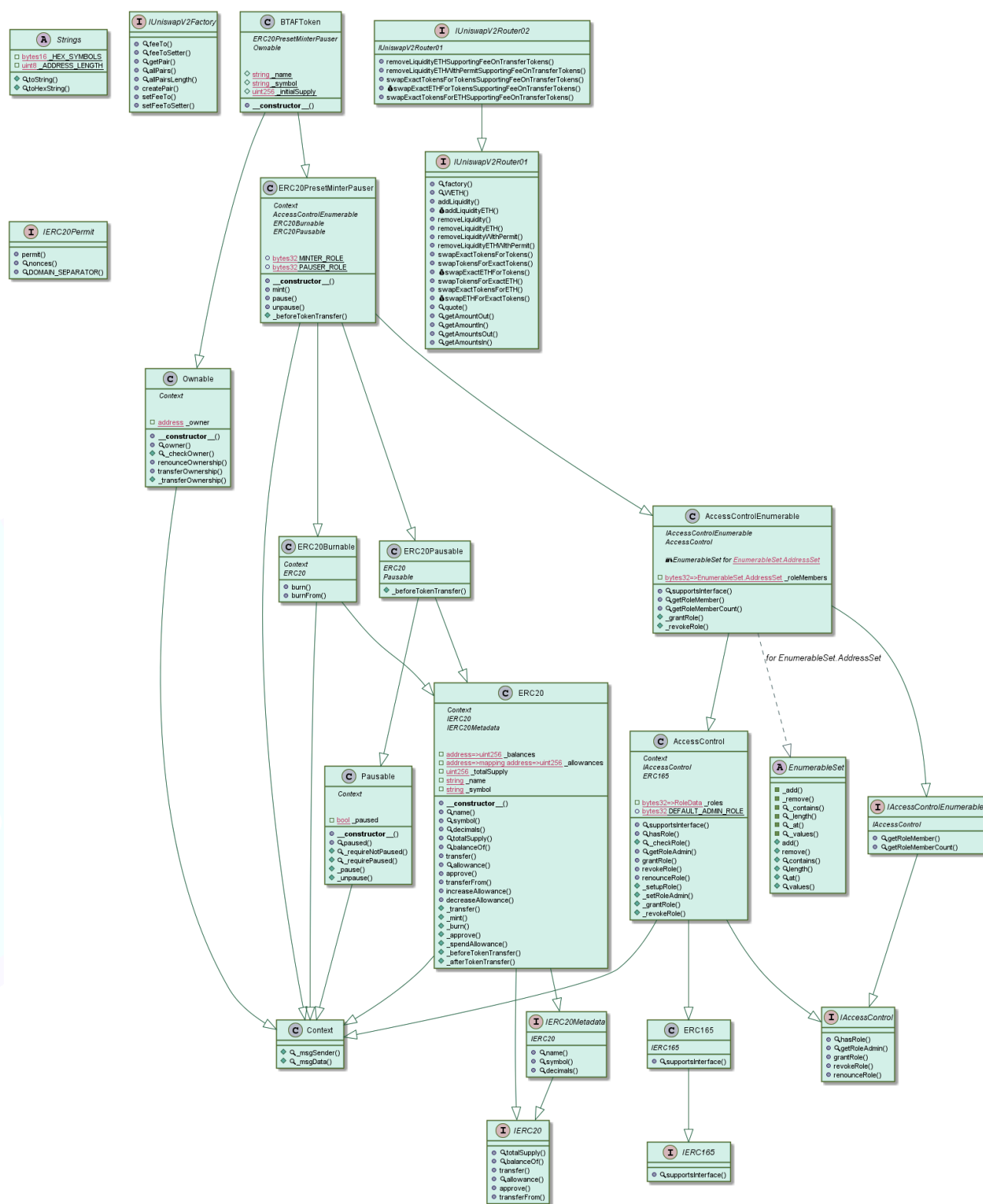
Website: hashlock.com.au

Contact: info@hashlock.com.au

Appendix

Code Flow Diagram - BTAF Token

BTAFToken Diagram



Slither Results Log

Slither Log >> BTAFToken.sol

```

INFO:Detectors:
ERC20PresetMinterPauser.constructor(string,string).name (BTAFToken.sol#808) shadows:
- ERC20.name() (BTAFToken.sol#584-586) (function)
- IERC20Metadata.name() (BTAFToken.sol#547) (function)
ERC20PresetMinterPauser.constructor(string,string).symbol (BTAFToken.sol#808) shadows:
- ERC20.symbol() (BTAFToken.sol#588-590) (function)
- IERC20Metadata.symbol() (BTAFToken.sol#549) (function)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#local-variable-shadowing
INFO:Detectors:
EnumerableSet.values(EnumerableSet.AddressSet) (BTAFToken.sol#206-215) uses assembly
- INLINE ASM (BTAFToken.sol#210-212)
EnumerableSet.values(EnumerableSet.UintSet) (BTAFToken.sol#242-251) uses assembly
- INLINE ASM (BTAFToken.sol#246-248)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage
INFO:Detectors:
Pragma version^0.8.4 (BTAFToken.sol#2) necessitates a version too recent to be trusted. Consider deploying with 0.6.12/0.7.6
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Function IUniswapV2Router01.WETH() (BTAFToken.sol#273) is not in mixedCase
Function IERC20Permit.DOMAIN_SEPARATOR() (BTAFToken.sol#566) is not in mixedCase
Constant BTAFToken._name (BTAFToken.sol#879) is not in UPPER_CASE_WITH_UNDERSCORES
Constant BTAFToken._symbol (BTAFToken.sol#880) is not in UPPER_CASE_WITH_UNDERSCORES
Constant BTAFToken._initialSupply (BTAFToken.sol#881) is not in UPPER_CASE_WITH_UNDERSCORES
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Detectors:
Variable IUniswapV2Router01.addLiquidity(address,address,uint256,uint256,uint256,uint256,address,uint256).amountADesired (BTAFToken.sol#278) is too similar to IUniswapV2Router01.addLiquidity(address,address,uint256,uint256,uint256,uint256,address,uint256).amountBDesired (BTAFToken.sol#279)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#variable-names-are-too-similar
INFO:Detectors:
grantRole(bytes32,address) should be declared external:
- AccessControl.grantRole(bytes32,address) (BTAFToken.sol#452-454)
revokeRole(bytes32,address) should be declared external:
- AccessControl.revokeRole(bytes32,address) (BTAFToken.sol#456-458)
renounceRole(bytes32,address) should be declared external:
- AccessControl.renounceRole(bytes32,address) (BTAFToken.sol#460-464)
getRoleMember(bytes32,uint256) should be declared external:
- AccessControlEnumerable.getRoleMember(bytes32,uint256) (BTAFToken.sol#505-507)
getRoleMemberCount(bytes32) should be declared external:
- AccessControlEnumerable.getRoleMemberCount(bytes32) (BTAFToken.sol#509-511)
name() should be declared external:
- ERC20.name() (BTAFToken.sol#584-586)
symbol() should be declared external:
- ERC20.symbol() (BTAFToken.sol#588-590)
decimals() should be declared external:
- ERC20.decimals() (BTAFToken.sol#592-594)
totalSupply() should be declared external:
- ERC20.totalSupply() (BTAFToken.sol#596-598)
balanceOf(address) should be declared external:
- ERC20.balanceOf(address) (BTAFToken.sol#600-602)
transfer(address,uint256) should be declared external:
- ERC20.transfer(address,uint256) (BTAFToken.sol#604-608)
approve(address,uint256) should be declared external:
- ERC20.approve(address,uint256) (BTAFToken.sol#614-618)
transferFrom(address,address,uint256) should be declared external:
- ERC20.transferFrom(address,address,uint256) (BTAFToken.sol#620-629)
increaseAllowance(address,uint256) should be declared external:
- ERC20.increaseAllowance(address,uint256) (BTAFToken.sol#631-635)
decreaseAllowance(address,uint256) should be declared external:
- ERC20.decreaseAllowance(address,uint256) (BTAFToken.sol#637-646)
burn(uint256) should be declared external:
- ERC20Burnable.burn(uint256) (BTAFToken.sol#739-741)
burnFrom(address,uint256) should be declared external:
- ERC20Burnable.burnFrom(address,uint256) (BTAFToken.sol#743-746)
pause() should be declared external:
- ERC20PresetMinterPauser.pause() (BTAFToken.sol#820-823)
unpause() should be declared external:
- ERC20PresetMinterPauser.unpause() (BTAFToken.sol#825-828)
renounceOwnership() should be declared external:
- Ownable.renounceOwnership() (BTAFToken.sol#861-863)
transferOwnership(address) should be declared external:
- Ownable.transferOwnership(address) (BTAFToken.sol#865-868)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#public-function-that-could-be-declared-external
INFO:Slither:BTAFToken.sol analyzed (22 contracts with 75 detectors), 57 result(s) found
INFO:Slither:Use https://crytic.io/ to get access to additional detectors and Github integration

```

Solidity Static Analysis

BTAFToken.sol

Security

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 246:8:

Gas & Economy

Gas costs:

Gas requirement of function BTAFToken.getRoleAdmin is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 448:4:

Gas costs:

Gas requirement of function BTAFToken.burnFrom is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 743:4:

Gas costs:

Gas requirement of function BTAFToken.mint is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 815:4:

Gas costs:

Gas requirement of function `BTAFToken.pause` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 820:4:

Gas costs:

Gas requirement of function `BTAFToken.unpause` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 825:4:

Miscellaneous**Constant/View/Pure functions:**

`Strings.toString(uint256)` : Is constant but potentially should not be. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 7:4:

Similar variable names:

`ERC20Burnable.burnFrom(address,uint256)` : Variables have very similar names "account" and "amount". Note: Modifiers are currently not considered by this static analysis.

Pos: 745:23:

Guard conditions:

Use "`assert(x)`" if you never ever want `x` to be false, not in any circumstance (apart from a bug in your code). Use "`require(x)`" if `x` can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 866:8:

Delete from dynamic array:

Using "`delete`" on an array leaves a gap. The length of the array remains the same. If you want to remove the empty position you need to shift items manually and update the "`length`" property.

[more](#)

Pos: 128:12:

Solhint Linter

BTAFToken.sol

```
BTAFToken.sol:641:18: Error: Parse error: missing ';' at '{'  
BTAFToken.sol:660:18: Error: Parse error: missing ';' at '{'  
BTAFToken.sol:689:18: Error: Parse error: missing ';' at '{'  
BTAFToken.sol:719:22: Error: Parse error: missing ';' at '{'
```

Software analysis result:

Our Automated software tools reported many false positive results and some informational issues. However, these issues can be safely ignored.

#Hashlock.



#Hashlock.

Hashlock Pty Ltd